

Algoritmos y Estructuras de Datos

Examen ordinario - Completo

Instrucciones

Lea atentamente las instrucciones siguientes antes de continuar:

- Para este examen **solo se permite el uso de material de escritura**, como bolígrafos, lápices, gomas o cinta correctora. No está permitido usar dispositivos electrónicos de ningún tipo, apuntes ni ningún otro material distinto del indicado. **No se corregirá ninguna parte del examen escrita en lápiz.**
- Si tiene más de una hoja, este examen se entrega grapado y **no deberá desgraparlo.**
- Los **enunciados** de los problemas incluyen **espacio para responder** en forma de cajas. **Solo se corregirán estos espacios.** Antes de continuar leyendo, por favor, escriba su nombre completo y DNI en todas las páginas que tengan cabecera.
- Se le proporcionará una hoja de sucio adicional.
- Al finalizar el examen deberá **tomar una fotografía de todos las páginas que tengan cabecera.**
- La correcta interpretación del enunciado se considera parte de las competencias a evaluar en el examen.
- Durante el examen aparecerán fragmentos de código, si en alguna parte de ellos aparece el texto "...", eso indicará que dicho fragmento en concreto no se proporciona porque no es relevante para la resolución del ejercicio.
- Cada ejercicio incluye todas las clases y métodos necesarios para resolverlo que no pertenezcan a las librerías estándar de Java.

Ya puede comenzar el examen. ¡Buena suerte!

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

Ejercicio 1: 1.5 puntos

Se dispone de un log de sistema en formato de texto. Cada línea representa un evento que sigue el formato: **ID_EVENTO:TIMESTAMP:MENSAJE**. De tal forma que:

- **ID_EVENTO**: Debe ser un número entero positivo.
- **TIMESTAMP**: Debe ser un número entero almacenado como long.
- **MENSAJE**: Texto libre.

Implemente el método estático `validarLog(String logContent)` que recibe el **contenido completo** del archivo. Debe **devolver true** si **todas** las líneas **cumplen el formato** y el **TIMESTAMP** de cada línea es **estrictamente mayor que el de la línea anterior (orden cronológico)**. Si hay errores de formato o el orden cronológico no se respeta, devuelva **false**. El método **no debe** lanzar excepciones.

Escriba su respuesta a continuación:

```
public static boolean validarLog(String logContent) {  
    String[] lineas = logContent.split("\n");  
    long ultimoTimestamp = -1;  
  
    for (String linea : lineas) {  
        String[] partes = linea.split(":");  
        if (partes.length < 3) return false; // Formato incorrecto  
        try {  
            // Validar ID (entero)  
            Integer.parseInt(partes[0]);  
            // Validar Timestamp  
            long actualTimestamp = Long.parseLong(partes[1]);  
  
            if (actualTimestamp <= ultimoTimestamp) return false; // No es cronológico  
  
            ultimoTimestamp = actualTimestamp;  
        } catch (NumberFormatException e) {  
            return false; // No son números válidos  
        }  
    }  
    return true;  
}
```

- División en líneas y en partes +25 %
- Validación del número de partes +25 %
- Validación de los timestamps +25 %
- Código correcto (incluyendo manejo de excepciones) +25 %

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

Ejercicio 2: 3 puntos

La clase `SignalProcessor` contiene un array de datos `float[] data`. Deberá desarrollar un algoritmo para encontrar el valor máximo dentro de un rango específico del array **usando recursividad**.

(2 puntos) Implemente el método `float findMaxRecursive(float[] data, int inicio, int fin)`. Este método debe dividir el rango en dos mitades, buscar el máximo en cada mitad recursivamente y devolver el mayor de los dos.

(1 punto) ¿Cuál es la complejidad computacional (en la notación O) de este algoritmo para un array de tamaño N ? Justifique su respuesta analizando cuántas veces se visita cada elemento o cuántas llamadas recursivas se realizan.

Escriba su respuesta a continuación:

```
private float findMaxRecursive(float[] data, int inicio, int fin) {  
    // Caso base  
    if (inicio == fin) {  
        return data[inicio];  
    }  
  
    // Paso recursivo  
    int mitad = (inicio + fin) / 2;  
    float maxIzq = findMaxRecursive(data, inicio, mitad);  
    float maxDer = findMaxRecursive(data, mitad + 1, fin);  
    if (maxIzq > maxDer) {  
        return maxIzq;  
    } else {  
        return maxDer;  
    }  
}
```

- Caso base +25 %
- Paso recursivo: llamadas recursivas correctas +25 %
- Paso recursivo: selección del valor correcto +25 %
- Código correcto +25 %

La complejidad es $O(N)$ (lineal). Aunque el código divide el problema por la mitad como en la Búsqueda Binaria, ambas mitades son procesadas. El algoritmo debe visitar (o comparar) cada uno de los N elementos del array al menos una vez para asegurarse de que tiene el máximo real. Si se prefiere, se puede entender que se trata de un árbol binario completo donde las hojas son los N elementos. Por tanto, el número de nodos de este árbol son $2N - 1$, lo que resulta en una complejidad $O(N)$.

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

Ejercicio 3: 3 puntos

Se desea modelar el funcionamiento de la zona de recogida de pedidos de un restaurante de comida rápida utilizando un monitor. El sistema cuenta con una clase **Bandeja** compartida con la que interactúan tres clases que extienden a **Thread**:

- **Cocinero**: Prepara hamburguesas y las deposita en la bandeja. Llamará al método `ponerHamburguesa()` de la clase **Bandeja**. Puede haber varios objetos de esta clase.
- **Cliente**: Toma las hamburguesas de la bandeja para consumirlas. Llamará al método `sacarHamburguesa()` de la clase **Bandeja**. Puede haber varios objetos de esta clase.
- **Limpiador**: Limpia la bandeja. Llamará al método `setLimpiando(boolean limpiando)` de la clase **Bandeja**.

La bandeja tiene una capacidad máxima de 5 hamburguesas. Si un **cocinero** intenta poner una hamburguesa y la bandeja está llena o está siendo limpiada, deberá esperar hasta que haya hueco o la limpieza haya terminado. Si un **cliente** intenta coger una hamburguesa y la bandeja está vacía o está siendo limpiada, debe esperar hasta que haya comida disponible o la limpieza haya terminado. La clase **Bandeja** tiene dos métodos para acceder y modificar el atributo boolean `limpiando`, que indica si la bandeja está siendo limpiada: `getLimpiando()` y `setLimpiando(boolean limpiando)`.

Implemente los métodos `getLimpiando()`, `setLimpiando(boolean limpiando)`, `ponerHamburguesa()` y `sacarHamburguesa()`. Debe garantizar la correcta sincronización de todas las hebras.

Escriba su respuesta a continuación:

```
public synchronized boolean getLimpiando() {
    return limpiando;
}

public synchronized void setLimpiando(boolean limpiando) {
    this.limpiando = limpiando;
    notifyAll(); // Despertar a cocineros y clientes esperando
}

public synchronized void ponerHamburguesa() throws InterruptedException {
    while (hamburguesas >= CAPACIDAD_MAX || limpiando)
        wait();
    hamburguesas++;
    notifyAll(); // Despertar a clientes esperando
}

public synchronized void sacarHamburguesa() throws InterruptedException {
    while (hamburguesas == 0 || limpiando)
        wait();
    hamburguesas--;
    notifyAll(); // Despertar a cocineros esperando
}
```

- Uso de `synchronized` +16,66 %
- Control del número de hamburguesas +16,66 %
- Condiciones de espera correctas +16,66 %
- `wait()` dentro de un `while` +16,66 %
- Notificación a las hebras en espera +16,66 %
- Código correcto +16,66 %

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

Ejercicio 4: 1.25 puntos

Se desea ampliar la última práctica de la asignatura para crear un servicio de búsqueda de pacientes. Cuando un usuario realiza una petición GET el servicio le muestra un formulario con una caja de texto para que introduzca el texto de búsqueda e.g. “os” y un botón para enviarlo (imagen izquierda). Cuando el servidor recibe los datos, comprobará si el nombre de alguno de los pacientes almacenados en el sistema incluye el texto buscado (tal y como se ha escrito), devolviendo en ese caso un mensaje que incluya de nuevo el formulario de búsqueda y una tabla con el nombre y apellidos de todos los pacientes que cumplen el criterio (imagen derecha, tras haber buscado los pacientes que contienen “os” en el nombre).

Introduce el término de búsqueda

Resultados de búsqueda

Introduce el término de búsqueda

Resultados de búsqueda
Marcos López
Petros Paraskevas

(0,25 puntos) Escriba a continuación el contenido del fichero buscar.html que se usará como plantilla en el servlet a desarrollar en el siguiente apartado. No hace falta que incluya estilos en este momento.

```
<!DOCTYPE html>
<html>
  <head></head>

  <body>
    <h1 id='i1'>Introduce el término de búsqueda</h1>
    <form action="buscar" method="POST" name="formulario">
      <input type="text" name="t">
      <input type="submit" value="Buscar">
    </form>
    <table>
      <tr><th>Resultados de búsqueda</th></tr>
      <tr></tr>
    </table>
  </body>
</html>
```

- Form correcto (action, method, etiquetas) +25 %
- Input correcto (input, submit) +25 %
- Mensajes y huecos correctos +25 %
- Todo el HTML completo y correcto +25 %

(1 punto) Escriba a continuación el código de un servlet que debe tratar las peticiones enviadas:

- Ante una petición HTTP GET, el servicio prepara la página buscar.html y la envía.
- Ante una petición HTTP POST, el servicio comparará el texto recibido con los nombres de la lista de pacientes que almacena, devolviendo:
 - Una tabla con una fila por cada paciente almacenado en el sistema cuyo nombre incluya el texto buscado, o
 - un mensaje en rojo si no se ha encontrado ningún paciente cuyo nombre incluya el término buscado.

Suponga que tiene disponible en su servlet el método `String getPlantilla (String fichero)` que, dado el nombre de un fichero disponible en su proyecto, devuelve el contenido de ese fichero como un `String`. Por ejemplo, `getPlantilla('alta.html')` devuelve el contenido del fichero `alta.html` de su proyecto como un `String`. Además, recuerde que dispone de un objeto `PacienteRepository` almacenado como atributo con nombre `repo` del `ServletContext` que guarda información de todos los pacientes dados de alta en su sistema. Este objeto `PacienteRepository` tiene un método `List<Paciente> getPacientes()` que devuelve el listado de pacientes.

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

```
// Fijar la ruta indicada en el enunciado y que coincida con la indicada
// en el action del form +25%
@WebServlet("/buscar")
public class Buscar extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Configurar la plantilla y enviar mensaje correcto +25%
        String html = getPlantilla("buscar.html");
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print(html);
        out.flush();
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String pagina = getPlantilla("buscar.html");
        // Recuperar el repositorio +25%
        PacienteRepository repo =
            (PacienteRepository) getServletContext().getAttribute("repo");
        // Recuperar el texto enviado por el form +25%
        String t = request.getParameter("t");
        // Recorrer pacientes (+25%), preparar mensaje si encontrado (+25%)
        boolean encontrados = false;
        for(Paciente p : repo.getPacientes()) {
            if(p.getNombre().contains(t)) {
                encontrados = true;
                pagina = pagina.replace("<tr></tr>", "<tr><td>" + p.getNombre() + " " +
                    p.getApellido() + "</td></tr><tr></tr>");
            }
        }
        // Gestionar si no encontrados (+25%) y devolver mensaje en rojo (+25%)
        if (!encontrados)
            pagina = pagina.replace("<tr></tr>", "<tr style='color:red;'>" +
                "<td>No se han encontrado pacientes con el término '" + t + "'<td></tr>");
        // Reemplazar en la plantilla y enviar (no puntúa, ya que se puntúa en el doGet)
        response.setContentType("text/html; charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print(html);
        out.flush();
    }
}
```

- Form correcto (action, method, etiquetas) +25 %
- Input correcto (input, submit) +25 %
- Mensajes y huecos correctos +25 %
- Todo el HTML completo y correcto +25 %

	APELLIDOS:	NOMBRE:	GRUPO:	NOTA:
	CURSO: Algoritmos y Estructuras de Datos	DNI:	FECHA: 22/01/26	

Ejercicio 5: 1.25 puntos

(0,50 puntos) Cree un servicio TCP de generador de números aleatorios que atiende peticiones en el puerto 1234 y, al recibir una línea con un número entero, devuelve un número aleatorio entre 0 y el número recibido. Nota: el método `int nextInt(int m)` de la clase `Random` devuelve un `int` entre 0 y `m`.

```
public static void main(String[] args) throws Exception {  
    // Levantar servidor y esperar conexión +25%  
    ServerSocket s = new ServerSocket(1234);  
    while(true) {  
        Socket r = s.accept();  
        // Obtener los streams +25%  
        BufferedReader in = new BufferedReader(new InputStreamReader(r.getInputStream()));  
        PrintWriter out = new PrintWriter(r.getOutputStream());  
        // Leer la línea que me envían y sacar el entero +25%  
        String str = in.readLine();  
        int i = Integer.parseInt(str);  
        // Generar y devolver el número +25%  
        Random r = new Random();  
        out.println(r.nextInt(i));  
        out.flush();  
        remote.close();  
    }  
}
```

(0,75 puntos) Ofrezca una respuesta corta a las siguientes cuestiones.

¿Puede un archivo CSS modificar el comportamiento del servidor web? Justifique su respuesta.

No. Un archivo CSS no puede modificar el comportamiento del servidor web, ya que el CSS es un recurso estático que se interpreta y aplica exclusivamente en el navegador del cliente. El servidor solo se limita a servir el archivo CSS solicitado y no ejecuta ni reacciona a su contenido.

Un estudiante afirma que “JavaScript se ejecuta en el servidor porque el servidor envía el código al navegador”.

¿Es correcta esta afirmación? Justifique su respuesta.

No, la afirmación es incorrecta. Que el servidor envíe el código JavaScript no implica que se ejecute en él. En una aplicación web clásica, el servidor se limita a entregar el archivo JavaScript como un recurso, mientras que su ejecución tiene lugar en el navegador del cliente.

Si un navegador ya ha descargado y mostrado un documento HTML, ¿por qué puede seguir enviando peticiones al servidor incluso sin interacción del usuario? Explique brevemente.

Porque el documento HTML puede incluir o activar procesos que generan nuevas peticiones de forma automática. Por ejemplo, la carga de recursos externos (imágenes, CSS, JavaScript) o la ejecución de código JavaScript que realiza peticiones periódicas o diferidas.

Algunos métodos disponibles en HttpServletRequest, junto con el tipo de valor devuelto	
Cookie []	getCookies() Devuelve un array con todas las cookies que el cliente ha enviado en esta petición.
String	getHeader(String name) Devuelve el valor de la cabecera indicada.
Enumeration<String>	getHeaders(String name) Devuelve todos los valores de la cabecera indicada.
String	getParameter(String name) Devuelve el valor del parámetro indicado, o null si no se ha recibido en la petición.
Enumeration<String>	getParameterNames() Devuelve los nombres de todos los parámetros recibidos en esta petición.
ServletContext	getServletContext() Devuelve un objeto que representa el contexto de ejecución.
HttpSession	getSession() Devuelve la sesión asociada con esta petición. Si no existe, crea una y la devuelve.
Algunos métodos disponibles en HttpServletResponse, junto con el tipo de valor devuelto.	
void	addCookie(Cookie c) Añade la cookie c a la respuesta.
void	addHeader(String name, String value) Añade la cabecera name a la respuesta con valor value.
OutputStream	getOutputStream() Devuelve un OutputStream para enviar bytes en la respuesta.
PrintWriter	getWriter() Devuelve un PrintWriter para escribir texto en la respuesta.
void	setContentType(String type) Fija el tipo de contenido de la respuesta.
Algunos métodos disponibles en HttpSession, junto con el tipo de valor devuelto	
Object	getAttribute(String name) Devuelve el objeto asociado a name en esta sesión, o null si no hay ningún objeto asociado a ese nombre.
Enumeration<String>	getAttributeNames() Devuelve todos los nombres asociados a los objetos almacenados en esta sesión.
void	removeAttribute(String name) Elimina el objeto asociado a name en esta sesión.
void	setAttribute(String name, Object value) Almacena el objeto value asociado al nombre name en esta sesión.
Algunos métodos disponibles en Cookie, junto con el tipo de valor devuelto.	
	Cookie(String name, String value) Crea una cookie con el valor especificado para name y value.
String	getName() Devuelve el nombre de esta cookie.
String	getValue() Devuelve el valor de esta cookie.
Algunos métodos disponibles en ServletContext, junto con el tipo de valor devuelto.	
Object	getAttribute(String name) Devuelve el objeto asociado a name en el contexto de ejecución del Servlet, o null si no está definido.
Enumeration<String>	getAttributeNames() Devuelve los nombres asociados a los objetos almacenados en este contexto.
void	removeAttribute(String name) Elimina el objeto asociado a name en este contexto.
void	setAttribute(String name, Object value) Almacena el objeto value asociado al nombre name en este contexto.